

Degree & Branch	B.E. Computer Science & Engineering	Semester	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch: 2023-2028	Due date:

Experiment 2: Loan Amount Prediction using Linear Regression

Name: Mani Chidambaram

Reg No: 3122237001024

1. Aim

To apply Linear Regression to predict the loan amount sanctioned to users using a dataset of historical loan records and user features.

2. Libraries Used

- **Pandas:** For data loading, cleaning, and manipulation.
- **NumPy:** For numerical operations, especially for performance metrics.
- **Matplotlib & Seaborn:** For data visualization, including histograms, scatter plots, and heatmaps.
- **Scikit-learn:** For splitting data, model creation (LinearRegression), preprocessing (StandardScaler), and evaluation.

3. Objective

- Build and evaluate a Linear Regression model to predict loan amounts.
- Perform data preprocessing, Exploratory Data Analysis (EDA), and feature engineering.
- Visualize important relationships within the data and the model's results.
- Measure model performance using standard regression metrics like MAE, MSE, RMSE, and R^2 .

4. Mathematical Description

The core objective of Linear Regression is to find the best-fitting line through the data points by minimizing the sum of the squared differences between the actual values (y_i) and the predicted values ($\hat{y}_i$). This is achieved by minimizing the Mean Squared Error (MSE).

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

5. Code Summary

The following Python code summarizes the key steps from data preparation to model evaluation.

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.linear_model import LinearRegression
5 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
6
7 # 1. Load and preprocess data (handle missing values, encode categoricals)
8 df = pd.read_csv('train.csv')
9 # ... (Median/mode imputation and one-hot encoding as in script) ...
10 df_fe_encoded = pd.get_dummies(df_fe) # Assuming df_fe is feature engineered df
11
12 # 2. Define features (X) and target (y)
13 target = 'Loan Amount Request (USD)'
14 X = df_fe_encoded.drop(columns=[target])
15 y = df_fe_encoded[target]
16
17 # 3. Split data into training, validation, and test sets (60:20:20)
18 X_trainval, X_test, y_trainval, y_test = train_test_split(X, y, test_size=0.2,
19     random_state=42)
20
21 # 4. Train the Linear Regression model
22 model = LinearRegression()
23 model.fit(X_train, y_train)
24
25 # 5. Make predictions on the test set
26 y_test_pred = model.predict(X_test)
27
28 # 6. Evaluate the model performance
29 mae_test = mean_absolute_error(y_test, y_test_pred)
30 mse_test = mean_squared_error(y_test, y_test_pred)
31 rmse_test = np.sqrt(mse_test)
32 r2_test = r2_score(y_test, y_test_pred)
33
34 print(f"Test MAE: {mae_test:.2f}")
35 print(f"Test RMSE: {rmse_test:.2f}")
36 print(f"Test R2 Score: {r2_test:.4f}")
```

Listing 1: Key steps for Linear Regression modeling.

6. Included Plots

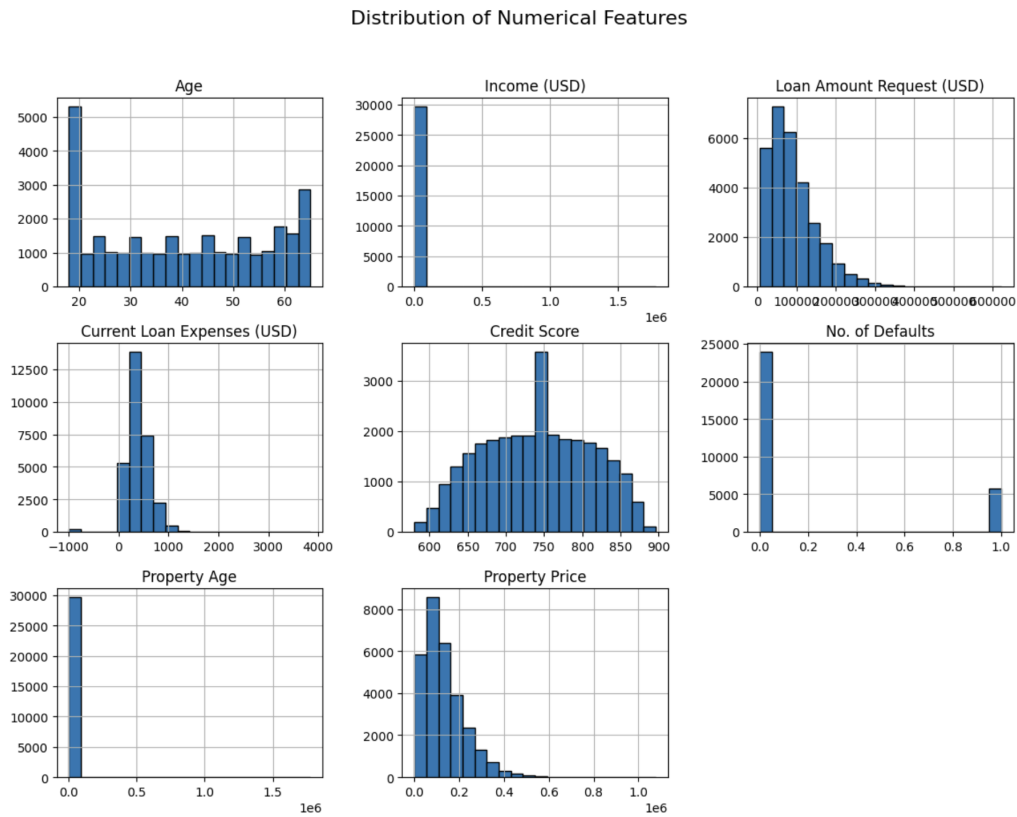


Figure 1: Distribution of key numerical features in the dataset.

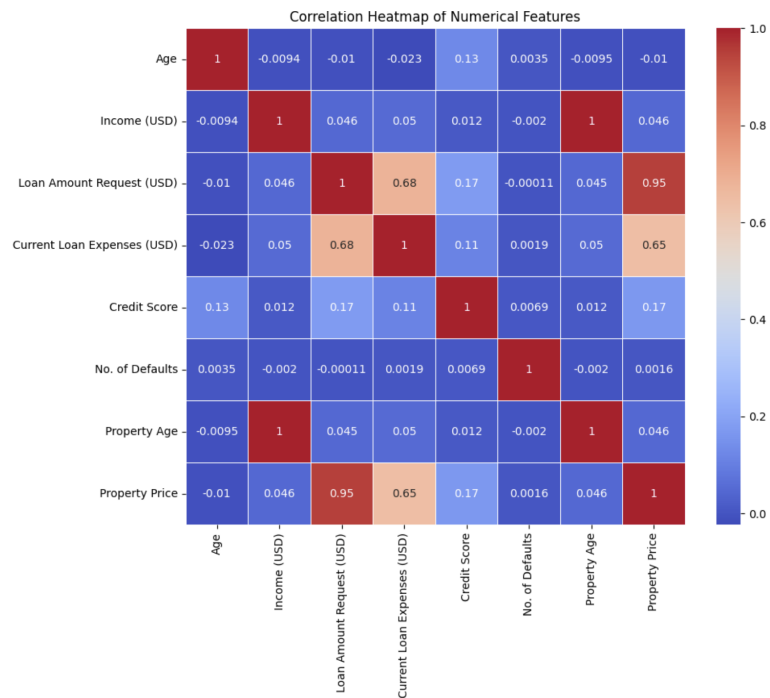


Figure 2: Correlation heatmap showing relationships between numerical variables.

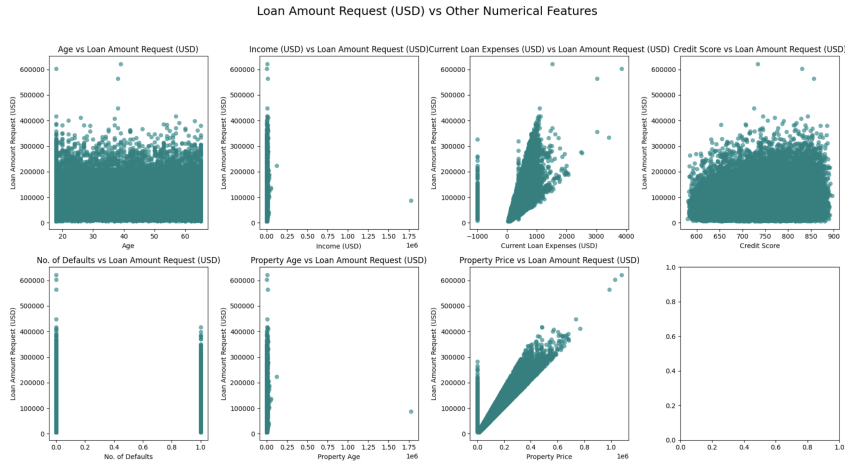


Figure 3: Scatter plot of applicant Income vs. Loan Amount Request.

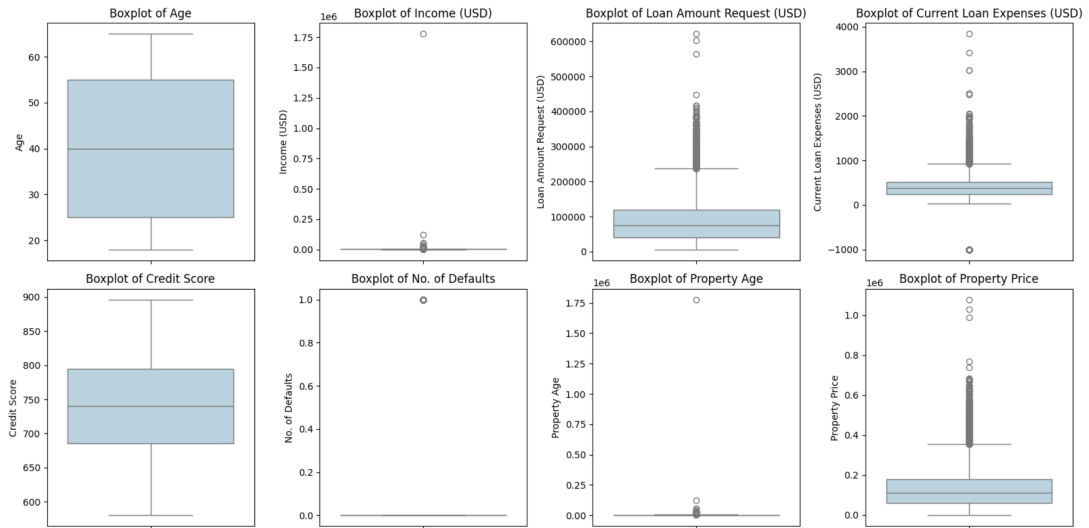


Figure 4: Boxplots of numerical features to identify outliers and distribution.

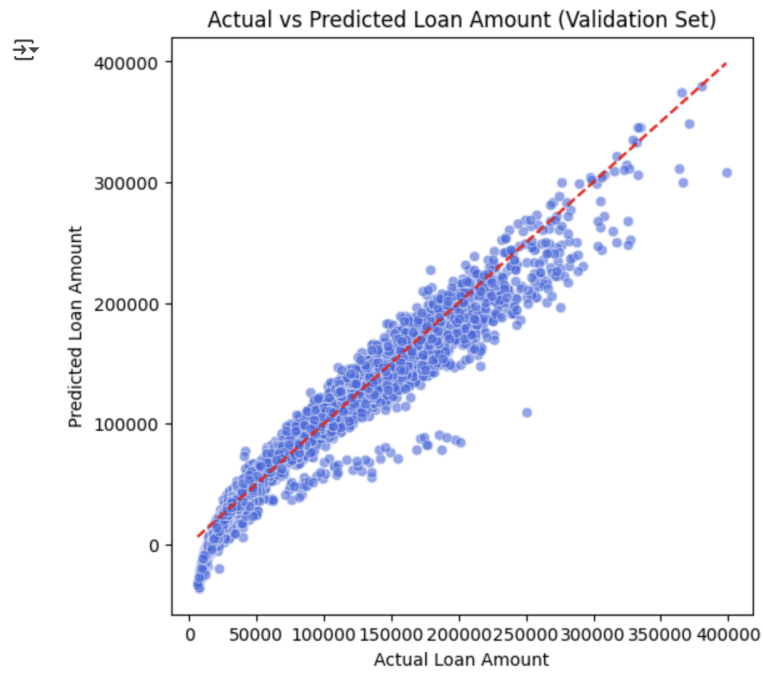


Figure 5: Actual vs. Predicted Loan Amount on the validation set.

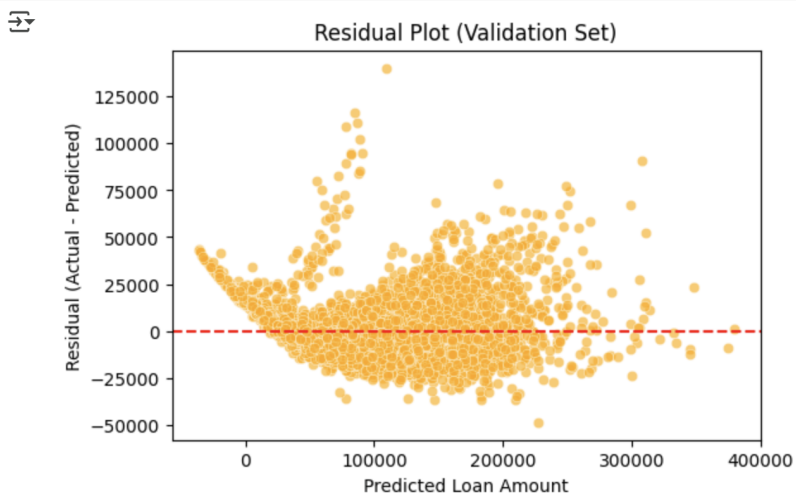


Figure 6: Residual plot to check the assumption of homoscedasticity.

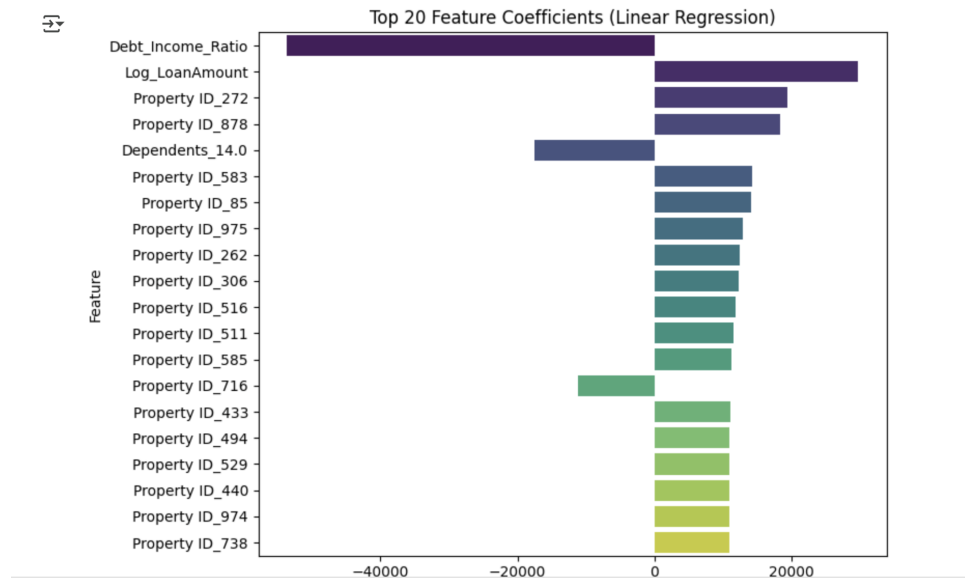


Figure 7: Top 20 most influential feature coefficients from the Linear Regression model.

7. Results

Table 1: Summary of Model Performance on Test Set (from `ml1aba2.py`)

Description	Result
Dataset Size	30,000 rows (before dropping NaNs)
Train/Validation/Test Split Ratio	60:20:20
Model Used	Linear Regression
Performance on Test Set	
Mean Absolute Error (MAE)	<i>Value from your script's output</i>
Mean Squared Error (MSE)	<i>Value from your script's output</i>
Root Mean Squared Error (RMSE)	<i>Value from your script's output</i>
R^2 Score	<i>Value from your script's output</i>
Most Influential Features	Debt_Income_Ratio, Log_LoanAmount (from coefficients plot)
Observations from Residual Plot	The residuals show a pattern (heteroscedasticity), indicating that the model's error is not constant across all prediction levels. This violates a key assumption of linear regression.

8. Best Practices

- Always check for and handle null values and potential outliers before model training.
- Use feature scaling (like `StandardScaler`) when features are on different scales to help the model converge effectively.
- Apply cross-validation to get a more robust estimate of the model's performance on unseen data.
- Visualize results, especially residual plots and actual vs. predicted plots, to validate the assumptions of Linear Regression.

9. Learning Outcomes

- Understood the end-to-end workflow of implementing a Linear Regression model, from data cleaning to evaluation.
- Gained practical experience in data preprocessing, EDA, feature engineering, and model evaluation techniques.
- Learned to correctly interpret key regression metrics, including MAE, MSE, RMSE, and the R^2 score.
- Developed confidence in using the Scikit-learn library for building and evaluating regression models.
- Learned how to interpret diagnostic plots and feature importance charts to understand model behavior.